

CSCE 637 COMPLEXITY THEORY · MODULE 09 · LECTURE NOTES

Circuits

A non-uniform model, and the most promising attack.

MODULE OUTCOMES

- Define circuit complexity and the class P/poly.
- Explain non-uniformity and what it buys.
- State the known circuit lower bounds and their limits.
- Explain the natural proofs barrier.
- Explain why circuits were the favoured attack.

1 The model, and why it was the favoured attack

A CIRCUIT FAMILY IS A NON-UNIFORM PROGRAM, WHICH IS STRICTLY MORE POWERFUL

A Boolean circuit computes a single function on a fixed number of input bits. A *family* provides one circuit per input length, and P/poly is the class of problems decided by families of polynomial size.

And crucially, nobody has to be able to *construct* the circuits — the family simply exists. That is the non-uniformity, and it is where the extra power comes from: a uniform model must produce its behaviour with one finite program, and a non-uniform one may have a different program for every length.

So P/poly contains undecidable problems. Any unary language is decided by a trivial circuit per length — accept or reject, constant — including a unary encoding of the halting problem. Which settles immediately that P/poly is not a model of anything you can run.

Which means P/poly is not a model of computation — it is a model of *circuit size* — and its value is twofold: lower bounds against it are correspondingly strong statements, and P is contained in it, which is what makes those lower bounds relevant (§1's second callout).

WHY CIRCUITS WERE THE FAVOURED ATTACK ON P VERSUS NP

P is contained in P/poly — convert the polynomial-time machine's computation on each input length into a circuit, which is the Cook–Levin tableau construction (Module 04 §2) read as a circuit. **So proving that SAT requires super-polynomial circuits would prove $P \neq NP$** , and it would prove something stronger.

And circuits are finite combinatorial objects with no computation, no time, no simulation, and no machine — which **evades the relativisation barrier** (Module 07 §2) **because there is nothing to attach an oracle to**. That was the hope, and it was a reasonable one.

And the technique worked on restricted models: exponential lower bounds were proved for monotone circuits, for bounded-depth circuits, and for formulas (§2's table) — real theorems, by real techniques, in the expected direction.

So for about a decade this looked like the route to $P \neq NP$. Then Razborov and Rudich proved in 1994 that the technique behind every one of those successes cannot be extended to general circuits (§3) — which is the second barrier, and the one that most changed the field's mood.

2 What has been proved

Model	Result	Note
Monotone circuits (AND and OR only, no negation)	Exponential lower bound for clique.	Razborov, 1985 — the result that started the optimism. And the restriction is severe: the same function has small non-monotone circuits in some related cases.
Bounded depth, unbounded fan-in (AC^0)	Parity requires super-polynomial size.	Håstad's switching lemma — unconditional, practically meaningful, and one of the field's genuine successes.
AC^0 with mod-p counting gates	Lower bounds known for appropriate functions.	Razborov–Smolensky, by approximating circuits with low-degree polynomials.
Formulas (circuits of fan-out 1)	Quadratic lower bounds for explicit functions.	Khrapchenko and successors.
General circuits	The best known bound is about $5n$ gates, for an explicit function.	Essentially nothing. See the callout.

THE GAP BETWEEN $5n$ AND EXPONENTIAL IS THE WHOLE PROBLEM

We believe SAT requires circuits of size $2^{\Omega(n)}$. We can prove about $5n$. That is the state of the art, and it is worth stating as two numbers rather than as a description.

And a counting argument shows that most Boolean functions require exponential-size circuits — there are 2^{2^n} functions on n bits and far fewer circuits of subexponential size, so almost all functions are hard.

So hard functions are abundant, and we cannot exhibit a single one — which is **exactly CSCE 627 Module 04 §4's counting-versus-construction distinction**, arriving in a different subject with the same structure: an easy existence proof and no constructive one.

Which is a useful calibration. The field's central conjecture is an exponential gap away from what it can prove, and knowing that number is considerably more informative than knowing the consensus — it is the most sobering figure in the course, and it should temper any sense that $P \neq NP$ is nearly established.

3 The natural proofs barrier

```
THE SUCCESSFUL LOWER BOUNDS all have one shape: find a
PROPERTY of Boolean functions such that
    the hard function HAS it
    and every small-circuit function does NOT
```

```
Razborov and Rudich called such a property NATURAL if it is
    CONSTRUCTIVE -- efficiently checkable from the
        function's truth table
    LARGE         -- a random function has it with
        non-negligible probability
```

Every known circuit lower-bound proof uses a property that is natural in this sense.

THE BARRIER: if strong one-way functions exist, then no natural property can separate P from NP.

WHY: a natural property is, by definition, an efficient test that distinguishes random functions from functions with small circuits. But a pseudorandom function generator -- which exists if one-way functions do -- produces functions that HAVE small circuits and that look RANDOM to every efficient test. So no such test can exist.

SO: the technique that proved every known lower bound cannot prove the one we want -- assuming exactly the kind of hardness we are trying to establish.

The barrier assumes the hardness it blocks you from proving, which is the self-referential sting and is what makes the result so striking — **if one-way functions do not exist then cryptography fails and P versus NP is probably settled the other way, so the assumption is one almost everyone in the field already makes**. And it is the same shape as CSCE 627 Module 05's proofs and Module 07 §2's **relativisation**: a technique's limits established by turning the technique on itself.

4 Where it stands, and the useful classes

KARP–LIPTON: NP IN P/POLY WOULD COLLAPSE THE HIERARCHY

If NP is contained in P/poly, then the polynomial hierarchy collapses to its second level (Module 05 §3).

So NP is believed not to be in P/poly, which means SAT is believed to require super-polynomial circuits even when non-uniformity is allowed — a strictly stronger belief than $P \neq NP$, since P/poly contains P and much else.

And this is Module 05 §4's reasoning style again: 'X would collapse the hierarchy' deployed as evidence that X is false — a conditional theorem used as a plausibility argument.

Which also means the circuit programme is attacking the right thing. A super-polynomial circuit lower bound for SAT is both sufficient for $P \neq NP$ and independently believed true, so the programme is not aiming at something that might be false — it is aiming at something nobody can prove.

- **AC^0 : constant depth, unbounded fan-in. Parity is provably not in it** — which is a real, unconditional, and practically meaningful limitation: no constant-depth circuit of polynomial size computes parity, whatever the technology.
- **NC: polylogarithmic depth, polynomial size. Efficiently parallelisable** (Module 06 §4) — **and the circuit view is the natural one for parallelism**, because depth is parallel time directly rather than by analogy.
- **NC^1 and the formula classes**, which correspond to bounded-width and expression-depth notions that appear in real hardware design and in compiler expression trees.
- **And circuit *depth* is latency while circuit *size* is area — which is why this model is the natural one for hardware** (CSCE 614), and why circuit complexity has an engineering readership independent of its theoretical one.
- **So the model has engineering content independent of the P versus NP programme, which is worth separating:** the AC^0 parity bound is a useful fact about what hardware can do in constant depth, and it is true regardless of what happens to the central question.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Arora & Barak — chapters 6, 14, and 23 (free draft)	Circuits, the known lower bounds, and the natural proofs barrier — chapter 23 covers §3 properly.
Razborov & Rudich — Natural Proofs (free)	§3's barrier, in the original — and the definitions of constructive and large are worth reading exactly.
Håstad — Computational Limitations of Small-Depth Circuits	§2's switching lemma and the parity lower bound, free from the author.

Resource	What it covers for this module
Aaronson — P =? NP, the circuit sections (free)	§2's calibration and §4's assessment, including the $5n$ figure and what it means.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Build a circuit for a small function** and count its gates and depth.
- 2 **Show that any unary language is in P/poly**, including an undecidable one.
- 3 **Explain why that means P/poly is not a model of computation.**
- 4 **Convert a polynomial-time algorithm into a circuit family** for a small input length, and verify the size is polynomial.
- 5 **Count the Boolean functions on 5 inputs** and the circuits of size 20, and conclude.
- 6 **Try to prove a super-linear lower bound** for an explicit function, and report where you get stuck.
- 7 **State the two conditions for a natural property** and check them against one known lower bound.
- 8 **Explain the natural proofs barrier** in your own words, including why the assumption is reasonable.
- 9 **State Karp–Lipton** and explain how it is used as evidence.
- 10 **Relate circuit depth to hardware latency** for a circuit you have seen in CSCE 614.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Define a circuit family and P/poly, and say why it is non-uniform.
- 2 Why does P/poly contain undecidable problems, and what does that mean?
- 3 Give three reasons circuits were the favoured attack.
- 4 Name five lower bound results and the model each is against.
- 5 What is the best general circuit lower bound, and what do we believe?
- 6 Why is the counting argument not enough?
- 7 Define a natural property and state the barrier.
- 8 Why is the barrier self-referential?
- 9 State Karp–Lipton and its use.
- 10 Name the practically meaningful circuit classes and the hardware connection.